

systems, software engineering, real-time systems, and pattern recognition. He has published over 90 technical papers in these and other areas. He is a Life Fellow of the Franklin Institute from which he received the Louis E. Levy Medal in 1963; he also received the Golden Plate Award of the American Academy of Achievement in 1964. He was the President of the IEEE Computer Society in 1974-1975, the Division V (Computer Society) Director of the IEEE in 1976-1977, and Chairman of IEEE Technical Activities Board Development Committees in 1979. He has been a Director of the American Federation of Information Processing Societies since 1972 and is now a member

of the Executive Committee. He was the Conference Chairman of the First Annual IEEE Computer Conference, Chicago, 1967, the General Chairman of the 1974 National Computer Conference, Chicago; and the General Chairman of the IEEE Computer Society's First International Computer Software and Applications Conference, Chicago, IL, 1977 (COMPSAC '77).

Dr. Yau is a member of Association for Computing Machinery, the Society for Industrial and Applied Mathematics, the American Association for the Advancement of Science, the American Society for Engineering Education, Sigma Xi, Tau Beta Pi, and Eta Kappa Nu.

On Deadlock Detection in Distributed Systems

VIRGIL D. GLIGOR AND SUSAN H. SHATTUCK

Abstract—A hierarchically organized and a distributed protocol for deadlock detection in distributed databases are presented in [1]. In this paper we show that the distributed protocol is incorrect, and present possible remedies. However, the distributed protocol remains impractical because “condensations” of “transaction-wait-for” graphs make graph updates difficult to perform. Delayed graph updates cause the occurrence of false deadlocks in this as well as in some other deadlock detection protocols for distributed systems. The performance degradation that results from false deadlocks depends on the characteristics of each protocol.

Index Terms—Deadlock detection, delayed graph updates, distributed systems, false deadlocks, on-line protocols, ostensibly blocked transactions, out-of-order graph updates.

I. INTRODUCTION

A **DISTRIBUTED SYSTEM** consists of a collection of processes which execute on spatially separated computers and which can communicate with one another by exchanging messages. Such a system differs from a traditional multiprocessing system in at least two important ways: first, there is no globally shared memory, and second, the delays in message delivery are not negligible when compared to the execution time of a single process.

In principle, protocols for deadlock detection in distributed systems need not differ from the more traditional protocols that are used in centralized systems. In practice, however, lack of shared memory makes the construction and the maintenance of various graphs used for deadlock detection the source of significant bookkeeping and, possibly, of performance problems. For example, fully distributed protocols for deadlock detection may require that each site of the distributed system (i.e., each computer installation) construct and maintain a

graph for deadlock detection [1], [2]. To ensure that each site maintains as accurate a view of the process and resource status as possible, per-site concurrency controllers exchange messages that cause additions and deletions of graph nodes and arcs. Arbitrary delays in message delivery may cause deletion requests for certain arcs to arrive at a site *before* the messages that would have caused the addition of those arcs to the local graph. “Out-of-order” graph updates do not appear in centralized systems because a single graph for deadlock detection is constructed and maintained in physically shared memory.

Most concurrency control mechanisms used in practice rely on resource locking and deadlock detection protocols to ensure consistency [3]. Renewed interest in distributed systems has resulted in the publication of at least ten protocols for deadlock detection. However, few of these protocols are correct and fewer appear to be practical. In this paper, we point out that, unlike their hierarchical protocol, the distributed protocol presented in [1] by Menasce and Muntz, is incorrect. We also point out further problems with this and with other deadlock detection protocols for distributed systems.

II. A COUNTEREXAMPLE TO THE MENASCE-MUNTZ DISTRIBUTED PROTOCOL

A. Preliminaries

In the Menasce-Muntz distributed protocol, *transactions* are required to lock resources before (and unlock them after) operations, such as read and write, are performed. Each transaction is associated with a site of the distributed system, and does not migrate from site to site after the transaction starts. A transaction can be thought of as a process that is equipped with locking and recovery mechanisms [3]. Lock requests and releases by transactions are arbitrated by a (concurrency) *controller*. The controller also builds the “transaction-wait-for” (TWF) graph and detects deadlocks. In distributed

Manuscript received June 14, 1979; revised December 12, 1979. This work was supported by the National Aeronautics and Space Administration under Contract NAS 5-24407.

The authors are with the Department of Computer Science, University of Maryland, College Park, MD 20742.

protocols for deadlock detection, *every* site has a controller. All controller's decisions to accept or reject lock requests are based solely on information kept at the controller's site.

In a TWF graph, there is a directed arc from transaction T' to transaction T'' if T' is *blocked* and must wait for T'' to release a lock on an object [1], [3]. A *nonblocked* transaction, which is waited for by another transaction, is represented by a node of the TWF graph with no outgoing arcs. The *blocking set* of a transaction T is the set of all nonblocked transactions which can be reached by following a directed path in the TWF graph starting at the node associated with transaction T . The node at which T is started, is called the *site of origin* of T or $\text{Sorig}(T)$; $\text{TWF}(k)$ is the TWF graph at site Sk .

The following two rules define the distributed deadlock protection protocol proposed in [1].

Rule 1: The resource R cannot be granted to T because it is being held by transactions $T_1, T_2, \dots, T_K$. Add an arc from transaction T to each of the transactions in the set $[T_1, T_2, \dots, T_K]$. If the addition of these arcs causes a cycle to be formed in $\text{TWF}(k)$, then a deadlock is detected, and an appropriate action is required for its resolution. For each transaction T' in blocking set (T), send the blocking pair (T, T') to $\text{Sorig}(T)$ if $\text{Sorig}(T) \neq Sk$ and to $\text{Sorig}(T')$ if $\text{Sorig}(T') \neq Sk$.

Rule 2: A blocking pair (T, T') is received. Add an arc from T to T' in $\text{TWF}(k)$. If a cycle is formed, then a deadlock exists, and must be resolved by an appropriate action. If T' is blocked and $\text{Sorig}(T') \neq Sk$, then for each transaction T'' in the blocking set(T'), send the blocking pair (T, T'') to $\text{Sorig}(T'')$ if $\text{Sorig}(T'') \neq Sk$.

B. The Counterexample

Assume the following *initial situation*: in a network consisting of three sites of the distributed system, each site Si , $1 \leq i \leq 3$, contains a resource Ri and a transaction Ti such that $\text{Sorig}(Ti) = Si$. Each resource Ri is available, and each transaction Ti is active (i.e., nonblocked).

Consider now the following sequence of events.

Event 1: The following requests occur in any order: Ti requests Ri at site Si , and the request is granted. (Note that each Ti is still active, but each Ri is locked.)

Event 2: T_1 requests R_2 at site S_2 , but the request cannot be granted. Rule 1 of the protocol is applied. Observe that:

- 1) the arc $T_1 \rightarrow T_2$ is added to the $\text{TWF}(2)$;
- 2) no deadlock is detected at site S_2 ;
- 3) since T_2 is still active, T_2 is the only transaction in the blocking set of T_1 at site S_2 . The blocking pair (T_1, T_2) is sent to site S_1 . Call this message m_1 .

Event 3: T_2 requests R_3 at site S_3 , but the request cannot be granted.

Rule 1 of the protocol is applied again. Similar observations as in 1)-3) above are made (except that index 2 is replaced by 3, and index 1 is replaced by 2, i.e., T_2 becomes T_3 , S_1 becomes S_2 , etc.).

Event 4: T_3 requests R_1 at site S_1 , but the request cannot be granted. Rule 1 of the protocol is applied. Observe that:

- 1) the arc $T_3 \rightarrow T_1$ is added to $\text{TWF}(1)$;

- 2) no deadlock is detected at site S_1 ;

- 3) the blocking set of T_3 at site S_1 contains only T_1 , since the message m_1 has not been received yet. The blocking pair (T_3, T_1) is sent to site S_3 . Call this message m_3 .

At this point in time, no deadlock is detectable at any site. Yet a global deadlock does in fact exist, since T_1 waits for T_2 at site S_2 , T_2 waits for T_3 at site S_3 , and T_3 waits for T_1 at site S_1 . This situation is illustrated in Fig. 1(a). The last event is as follows.

Event 5: Messages m_1 , m_2 , and m_3 are received in any order. (For this example we assume that they are received in the order they are sent.)

- 1) $m_1 = (T_1, T_2)$ is received at site S_1 . Rule 2 of the protocol is applied now, and arc $T_1 \rightarrow T_2$ is added to $\text{TWF}(1)$. No deadlock is detected at site S_1 . Since T_2 is unblocked, and $\text{Sorig}(T_1) = S_1$, no messages are generated.

- 2) Similarly, when messages m_2 and m_3 arrive at site S_2 and S_3 , respectively, Rule 2 is applied, no deadlock is detected, and no further messages are generated.

At this point, all deadlock detection activity has ceased, but the global deadlock has not been detected because it is not observable in any local TWF graph. The TWF graphs are shown in Fig. 1(b). Deadlock remains undetected in the above example because the determination as to whether a transaction is blocked is incorrect. This determination cannot be made until the outcome of a request for a nonlocal resource is known. After a transaction issues a request for a nonlocal resource, that transaction is actually waiting until the response to that request arrives. Furthermore, when sufficient information is obtained to determine correctly whether a transaction is blocked (i.e., when a blocking pair arrives), the protocol fails to use that information.

For example, in Event 4, part 3), transaction T_1 is designated as nonblocked, by definition. That is, there is no outgoing arc from node T_1 of the $\text{TWF}(1)$ graph. The controller at site S_1 cannot discover that T_1 is blocked by T_2 at site S_2 merely by inspecting $\text{TWF}(1)$ graph. Furthermore, when message $m_1 = (T_1, T_2)$ arrives at site S_1 (in Event 5), Rule 2 fails to generate a new blocking pair which would show that transaction T_3 is waiting (indirectly) for transaction T_2 . Rule 2 only looks "forward" to the blocking set of T_1 for new information, and therefore, it ignores the fact that transaction T_3 waits (indirectly) for transaction T_2 .

In order for site S_1 (i.e., the controller in site S_1) ever to detect the global deadlock from its own TWF graph, it must receive a blocking pair from another site showing T_2 waiting for T_3 , or T_1 waiting indirectly for T_3 , or T_2 waiting indirectly for T_1 . But neither of the blocking pairs (T_2, T_3) , (T_1, T_3) , (T_2, T_1) will ever be received at site S_1 . In Event 5, Rule 2 fails to detect (and thus transmit) the new information available at site S_2 , when message m_2 is received, and at site S_3 , when message m_3 is received.

C. Possible Remedies

Two alternative approaches can be used to correct the protocol. The first approach is based on two requirements. First, a site must determine precisely whether a transaction at that site is 1) active, 2) blocked, or 3) waiting for the outcome of a

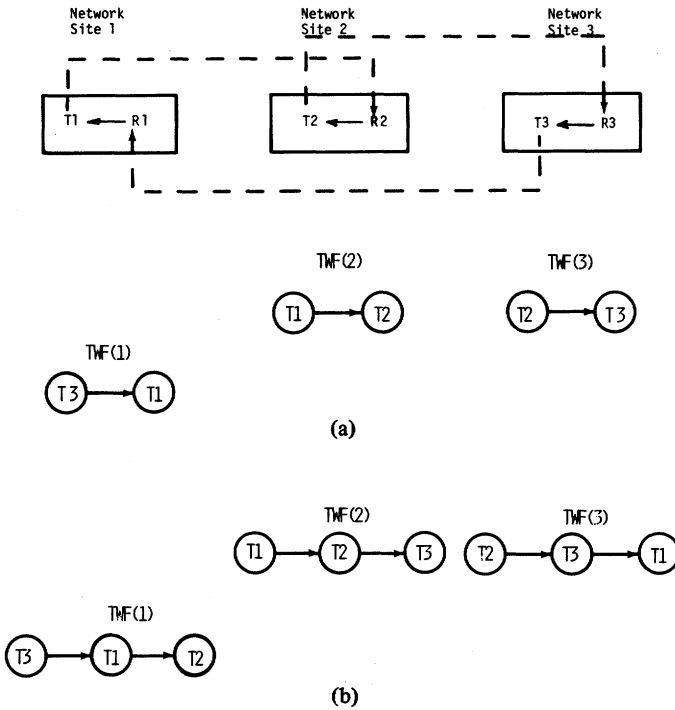


Fig. 1. Undetected deadlock and TWF graphs. (a) TWF graphs after Event 4. (b) TWF graphs after Event 5. $\rightarrow$ denotes a denied lock request. $\leftarrow$ denotes a locked resource.

request for a nonlocal resource. (This determination is made possible by the underlying assumption of the transaction model: a transaction does not have multisite incarnations and does not migrate from site to site.) Second, when it becomes known that a waiting transaction is blocked, Rule 2 must also look "backwards." That is, Rule 2 must inform the site of origin of various transactions (whose blocking sets include the newly blocked transaction) that they must update their local graphs with new blocking pairs.

Based on these observations, D. Mensasce [4] suggests the following additional definitions and version of the distributed protocol.

The *potential-blocking set* of a transaction T is the set of all waiting transactions that can be reached from T . A pair (T, T') is a *potential-blocking pair* if T' belongs to the potential-blocking set of T . The new rules of the protocol read as follows:

Rule 0: When transaction T requests a nonlocal resource, it is marked as a waiting transaction.

Rule 1: The resource R cannot be granted to T because it is being held by transactions $T_1, T_2, \dots, T_K$. Add an arc from transaction T to each of the transactions in the set $[T_1, T_2, \dots, T_K]$. If the addition of these arcs causes a cycle to be formed in $TWF(k)$, then a deadlock is detected, and an appropriate action is required for its resolution. For each transaction T' in blocking set (T) , send the blocking pair (T, T') to $Sorig(T)$ if $Sorig(T) \neq Sk$ and to $Sorig(T')$ if $Sorig(T') \neq Sk$. Form a list of potential-blocking pairs associated with T .

Rule 2: A blocking pair (T, T') is received. Add an arc from T to T' in $TWF(k)$. If a cycle is formed, then a deadlock exists, and must be resolved by an appropriate action.

Rule 2.1: If T' is blocked and $Sorig(T) \neq Sk$, then for

each transaction T'' in the blocking set (T') , send the blocking pair (T, T'') to $Sorig(T'')$ if $Sorig(T'') \neq Sk$.

Rule 2.2: If T is waiting and $Sorig(T) = Sk$ then, for each potential-blocking pair (T'', T) , send the blocking-pair (T'', T') to $Sorig(T'')$ if $Sorig(T'') \neq Sk$. Then, discard the potential-blocking pairs (T'', T) and erase the "waiting" mark of T .

An alternative approach is to add to Rule 2 the requirement that whenever a blocking pair (T, T') is received, the closure edges (T_0, T') must be generated for each $(T_0, T) \in TWF(k)$, and must be sent to $Sorig(T_0)$ if $Sorig(T_0) \neq Sorig(T)$. Thus, Rule 2 is made to "look backwards," and to extend TWF graphs at their "blocked" ends, whenever a blocking pair arrives. Although this alternative modification of the protocol is logically simpler than the previous one, it certainly adds a significant number of messages that must be passed around.

It is easy to verify that the two alternative modifications correct the protocol in the case of the counterexample. The proof that the modified protocol is correct appears to be more complex, however. Providing formal proofs of correctness for the modified versions of this protocol, is not as fruitful an exercise as one would hope, since the distributed protocol remains impractical.

III. IMPLEMENTATION CONSIDERATIONS

Most implementation considerations for deadlock detection protocols are based on estimates of the amount of intersite communication activity (or lack thereof) required to construct and maintain data structures for deadlock detection. Although most protocols construct and maintain list- or graph-like data structures, the implementation complexity required to maintain such data structures differs from protocol to protocol.

The maintenance of the TWF graphs (i.e., node and arc deletions) that are constructed by the Menasce-Muntz distributed protocol is not as straightforward as it might first appear. Several difficulties are encountered due to the fact that the protocol generates "condensations" of the TWF graphs [1], rather than the actual TWF graphs as defined in the previous section. The condensed TWF graphs do not necessarily include all blocked transactions, nor do they adequately reflect the relative positions of the waiting transactions within the graphs.

For example, consider the chain of waiting transactions $T_1 \rightarrow \dots \rightarrow T_{i-1} \rightarrow T_i \rightarrow T_{i+1} \rightarrow \dots \rightarrow T_{n-1} \rightarrow T_n$. When transaction T_n , at the end of the chain, releases a resource and allows T_{n-1} to become unblocked, the arcs $T_{n-1} \rightarrow T_n$ must be deleted from all sites that include node T_n . If the chain of waiting transactions were generated in its entirety at the site of origin of each $T_i, i = 1, \dots, n$, then the deletion request for arc $T_{n-1} \rightarrow T_n$ and for node T_n could be broadcast by the controller at $Sorig(T_n)$ to all sites containing the chain. To perform this broadcast, the controller at $Sorig(T_n)$ would have to keep a list of all sites where node T_n appears. In general, each controller would have to have a list of all sites which have transactions waiting for resources managed by that controller.

However, since the above chain may *not* appear in its en-

tirety in TWF graphs, the controller at $\text{Sorig}(T_n)$ must perform the following two additional steps in order to generate the new blocking transaction of the chain. First, the controller must determine the *real* $T_n - 1 \rightarrow T_n$ arc out of the set of direct and indirect arcs $T_j \rightarrow T_n$, $j = 1, \dots, n - 1$. The relative position of node $T_n - 1$ versus nodes T_j is thus established. Second, the controller must request the change of indirect arcs $T_j \rightarrow T_n$ into arcs $T_j \rightarrow T_n - 1$, $j = 1, \dots, n - 2$, at all sites where node $T_n - 1$ appears in the chain; moreover, the controller must request the replacement of indirect arcs $T_j \rightarrow T_n$ with arcs $T_j \rightarrow T_n - 1$, $j = 1, \dots, n - 2$, at all sites that do not include node $T_n - 1$ in the chain (due to the use of indirect arcs). To perform these additional steps, the controller must not only distinguish between direct and indirect arcs, but it must also generate new blocking pairs. Consequently, the protocol must be changed so that receipt of new blocking pairs during arc and node deletions should not initiate further construction of new blocking pairs, etc. The changes that are necessary to perform these additional steps result in increased protocol complexity.

Should the new blocking transaction $T_n - 1$ not be generated in some of the TWF graphs, and should node T_n be simply deleted, then deadlocks would undoubtedly remain undetected. Some of the T_j , $j = 1, \dots, n - 2$, would appear unblocked, even though they would actually be waiting on $T_n - 1$. Alternatively, should T_n remain a blocking node, then some T_j would be *ostensibly blocked* (i.e., T_j would be active but would appear blocked in some TWF graphs). Thus, some TWF graphs would inaccurately reflect the status of some transactions. The implications of this anomaly, and other cases in which it appears, are discussed below.

A more complicated situation occurs when transaction T_i , in the middle of the above chain, releases a resource that allows transaction $T_i - 1$ to become unblocked. Such a resource release occurs whenever transaction T_i aborts, or whenever transaction T_i is picked as the backup victim during recovery from deadlock. In this case, it is not sufficient to merely delete the arcs that include node T_i . Indirect arcs $T_i - 1 \rightarrow T_k$, $k = i + 1, \dots, n$, must be deleted, also. Moreover, every indirect arc $T_j \rightarrow T_k$, $j = 1, \dots, i - 2$, and $k = i + 1, \dots, n$, must be changed to an arc $T_j \rightarrow T_i - 1$, since transaction $T_i - 1$ becomes the blocking transaction for all T_j 's. However, because the sites which contain arcs that intervene between T_j and T_k are *unknown*, every indirect-arc change must be sent to *all* network sites. Thus, cascading of arc changes occurs whenever a site receives a request to change its TWF graph, and that site must request changes in TWF graphs of other sites, etc. The cascading of arc changes generates a flood of messages that can severely impair the system's operation. Should these arc changes not be performed, then some transactions would certainly become ostensibly blocked frequently.

A further complication also appears in some other protocols that 1) construct deadlock detection graphs and that 2) attempt to maintain those graphs as the distributed system runs. As mentioned in the introduction, graph-update messages may arrive out of order, due to network delays. Thus, a message requesting the deletion of an arc may be received before the message containing the request (e.g., the blocking pair) that

would generate that arc. If deletion requests for arcs that are not yet created are buffered by local controllers, then buffer management facilities must be added to these deadlock detection protocols. If such deletion requests are simply discarded, then ostensibly blocked transactions would undoubtedly appear. Subsequent deletion of obsolete arcs becomes necessary to make TWF graphs reflect the correct transaction status.

Ostensibly blocked transactions result from the inability to maintain an accurate view of the transaction status, and have two implications. First, they can lead to *false deadlocks* (i.e., deadlocks that do not actually exist, but that are "detected" anyway), and consequently, to unnecessary transaction backup. Second, they cause a large number of unnecessary blocking pairs to be sent to various sites. These blocking pairs compound the problem by causing the generation of more blocking pairs, and possibly, by causing the occurrence of false deadlocks.

It must be noted that ostensibly blocked transactions and false deadlocks are anomalies that occur in most deadlock detection protocols for distributed systems. However, the frequency with which these anomalies occur, and the extent of their consequences, depend on the particular protocol and on the communication delays. As an example, we show below that whenever distributed systems use centralized protocols for deadlock detection, it is fairly simple to discover that a deadlock is a false one, and to correct the global TWF graph. By contrast, if distributed protocols are used, it can be prohibitively expensive 1) to determine whether a deadlock is false and 2) to bring each TWF graph up-to-date at every site. To distinguish between a false deadlock and a real one may require $O(n^2)$ messages to be exchanged among n sites. Whenever incomplete versions of the TWF graphs are maintained at various sites (as in the case of the distributed protocol of Menasce and Muntz), the removal of false deadlocks, and the reconstruction of the actual TWF graph at every site, becomes a complex task. In the worst case, all resource allocation must be stopped, all TWF graphs must be gathered at each site, and each site must generate a new TWF graph that reflects the real transaction status and position within the graph.

If false deadlocks are infrequent, then the obvious alternative would be to accept unnecessary transaction backup. However, to avoid frequent false deadlocks, the Menasce-Muntz distributed protocol requires the complex, costly updates mentioned above. For this reason we believe that this distributed protocol is impractical.

IV. OTHER DEADLOCK DETECTION PROTOCOLS

In this section we present briefly the important aspects of other deadlock detection protocols which are relevant to the current discussion. For a more detailed presentation of the other deadlock detection protocols the reader is referred to [2] - [7].

A. Decentralized Protocols

An interesting decentralized protocol for deadlock detection that does not rely on continuous maintenance of TWF graphs is presented by Goldman in [5]. This protocol requires the construction of an ordered blocked-process list

(OBPL) every time deadlock detection is initiated. The OBPL denotes a group of processes each of which waits to access a resource that is held by the next process in the list. The last entry of the list identifies a process that either waits for, or controls, a specially identified resource. If the last process waits for that resource, then either a deadlock exists (i.e., a cycle is formed in the list) or, alternatively, the OBPL must be expanded further. Whenever the specially identified resource is nonlocal, the OBPL is sent to the site that controls the resource for further expansion.

The repeated construction of the OBPL, and the repeated checking of the last-process status, makes Goldman's protocol less susceptible to false deadlocks than any protocol that maintains TWF graphs. However, false deadlocks are not completely eliminated. Multiple copies of an OBPL are made whenever any process waits for a shared (e.g., readable) resource. Consequently, abortions of some processes within some OBPL's might not be reflected in the other OBPL copies soon enough to prevent the occurrence of false deadlocks. Although Goldman's protocol is somewhat restrictive because it prevents a process from issuing multiple resource requests, it is the only correct decentralized protocol to date (i.e., December 1979). The cost of repeated construction of OBPL's may result in infrequent, periodical initiation of deadlock detection, however.

The "on-line" protocol proposed by Isloor and Marsland [2] has as a main objective the timely detection of deadlock at the same site that causes the deadlock. This protocol differs from the distributed one proposed by Menasce and Muntz in that it maintains complete transaction (process)-resource graphs—not just condensed TWF graphs—at every site of the distributed system. Therefore, this protocol must broadcast graph updates to all sites whenever any resource allocation decision is made—not just periodically, nor only when transactions are blocked or resources are released.

The "on-line" protocol is also incorrect because deadlock detection is not initiated when a site receives a request to add a graph arc from a different site [6]. Although this problem can be easily corrected, it is not clear that the advantages of early detection of deadlocks offset the cost of using this protocol (e.g., broadcast of every graph update throughout the system, frequent initiation of detection activity, etc.). However, false deadlocks, which can still appear, can be handled fairly easily. Whenever a site controller detects a deadlock, it waits for a finite period of time (i.e., no greater than the maximum message delay) before it initiates recovery. If the deadlock persists, it is real. Removing false deadlocks in this manner would run counter to the intent of the protocol, however, because the detection of deadlocks is no longer immediate.

B. Centralized and Hierarchical Protocols

In centralized deadlock detection, one of the site controllers is designated as responsible for the detection of network-wide deadlocks [3], [5]. Periodically, or whenever a wait on an external resource occurs, the central controller constructs a global, complete TWF graph (or an OBPL [5]) using information received from local controllers. Then, the central controller initiates the detection of global deadlocks.

Centralized deadlock detection is conceptually simple. Moreover, some practical problems, such as the removal of false deadlocks, are easily solved. For example, whenever a global deadlock is detected, the central controller can reconstruct the deadlock cycle using new information received from local controllers. If the deadlock cycle persists, then the deadlock is real. However, the design of efficient, centralized deadlock detection protocols does not appear to be a simple task despite their conceptual simplicity. For example, Sun illustrates in [7] a subtle timing problem which renders Goldman's approach to centralized deadlock detection incorrect.

The two major sources of practical difficulties with centralized deadlock detection are 1) the high cost of constructing the TWF graph whenever a transaction waits on a global resource and 2) the vulnerability to failures of the central site. The impact of the high detection costs on the system performance can be attenuated by performing deadlock detection only periodically. However, the length of the "optimal" period (if one exists) cannot be easily determined without careful experimentation. The vulnerability to failures of the central site can be decreased by designating the controllers of several sites as central controllers. Along this line, the hierarchical protocol of Menasce and Muntz [1] appears to be an interesting alternative.

V. CONCLUSIONS

Some anomalies, such as the occurrence of false deadlocks and out-of-order requests for graph updates, are presented in the context of the distributed protocol for deadlock detection proposed in [1]. To some extent, these anomalies also appear in other distributed protocols. Distributed protocols can be more resistant to site failures than centralized ones; however, they are more vulnerable to the occurrence of false deadlocks. Centralized protocols are more attractive for practical purposes because of their simplicity. Moreover, careful design can attenuate, and sometimes remove, most difficulties of centralized deadlock detection. Further performance evaluation will clarify the relative merits of the protocols discussed above.

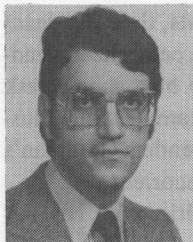
ACKNOWLEDGMENT

We thank B. Lindsay and T. Minoura for their comments on an earlier draft of this paper.

REFERENCES

- [1] D. Menasce and R. Muntz, "Locking and deadlock detection in distributed data bases," *IEEE Trans. Software Eng.*, vol. SE-5, pp. 195-202, May 1979.
- [2] S. S. Isloor and T. A. Marsland, "An effective 'on-line' deadlock detection technique for distributed data base management systems," in *Proc. COMSAC 1978*, pp. 283-288.
- [3] J. N. Gray, "Notes on data base operating systems," in *Operating Systems—An Advanced Course*, vol. 60, Bayer, Graham, and Segmuller, Eds. New York: Springer-Verlag, 1978, pp. 394-481.
- [4] R. Muntz, personal communication, Oct. 1979.
- [5] B. Goldman, "Deadlock detection in computer networks," M.I.T., Cambridge, Tech. Rep., M.I.T.-LCS, TR-185, Sept. 1977.
- [6] C. Mohan, "Distributed data base management—Progress, problems, some proposals and future directions," Software and Data base Eng. Group, Dep. Comput. Sci., Univ. Texas at Austin, working paper WP-7802, May 1979.

- [7] H. C. Sun, "Deadlock detection in distributed systems—A preliminary evaluation," Masters report, Dep. Comput. Sci., Univ. Maryland, 1978.



Virgil D. Gligor received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering and computer science in 1972, 1973, and 1976, respectively, all from the University of California, Berkeley.

In 1976, he joined the Department of Computer Science, University of Maryland, College Park, as an Assistant Professor. Between 1977 and 1980 he was also associated with the Computer Systems Group of Burroughs Corporation. He has lectured in computer science at the University of California at Santa Barbara (1974–1976) and at the Institutul

Politechnic Bucuresti in Romania (1972). His research interests include reliability and integrity of operating systems, computer architecture, and distributed database systems.

Politechnic Bucuresti in Romania (1972). His research interests include reliability and integrity of operating systems, computer architecture, and distributed database systems.



Susan H. Shattuck received the B.Sc. degree from Duke University, Durham, NC, in 1967 and the M.Sc. degree from the University of Maryland, College Park, in 1977.

Since 1974 she has worked on various research projects in the Department of Computer Science, University of Maryland. In 1967–1969 she was a Systems Programmer for the Naval Command Systems Support Activity, Washington, DC. Her research interests include distributed operating systems and reliability and integrity of operating systems.

liability and integrity of operating systems.

Performance Evaluation of Asynchronous Concurrent Systems Using Petri Nets

C. V. RAMAMOORTHY, FELLOW, IEEE, AND GARY S. HO, MEMBER, IEEE

Abstract—Some analysis techniques for real-time asynchronous concurrent systems are presented. In order to model clearly the synchronization involved in these systems, an extended timed Petri net model is used. The system to be studied is first modeled by a Petri net. Based on the Petri net model, a system is classified into either: 1) a consistent system; or 2) an inconsistent system. Most real-world systems fall into the first class which is further subclassified into i) decision-free systems; ii) safe persistent systems; and iii) general systems. Procedures for predicting and verifying the system performance of all three types are presented. It is found that the computational complexity involved increases in the same order as they are listed above.

Index Terms—Asynchronous, concurrent, performance, Petri net, real time.

I. INTRODUCTION

THE RECENT advances in solid-state technology have provided computer designers with powerful functional capabilities at low cost. This enables computer designers to isolate the functions of a computing element and add intelligence to functional units that can optimize their circuits locally. Such an approach eliminates the cost involved in time and band-

width to submit these local events for a central judgment in real time. This trend has led to the emergence of distributed systems. Furthermore, the recent growth in microprocessor architectures, their capabilities, and their low cost, have motivated system designers to design computer systems as distributed networks of microprocessors. By using multiple processing elements, system throughput can be improved and processing requirements and capabilities unobtainable by uniprocessors can be satisfied. However, the success of multiple processor systems greatly depends on the effectiveness of the synchronization among the processing elements.

In this paper, we concentrate our discussion on techniques for the prediction and verification of performance of distributed systems. We consider a distributed system as a set of loosely or tightly coupled processing elements working cooperatively and concurrently on a set of related tasks. In general, there are two approaches for performance evaluation [6], [2]: 1) deterministic models and 2) probabilistic models. In deterministic models, it is usually assumed that the task arrival times, the task execution times, and the synchronization involved are known in advance to the analysis. With this information, a very precise prediction of the system performance can be obtained. This approach is very useful for performance evaluation of real-time control systems with hard deadline requirements.

In probabilistic models, the task arrival rates and the task service times are usually specified by probabilistic distribution functions. The synchronization among tasks is usually not

Manuscript received February 00, 1979; revised December 00, 1979. This work was supported by the Ballistic Missile Defense Advanced Technology Center under Contract DASG60-77-C-0138, the U.S. Air Force under Grant F49620-79-C-0173, and the National Science Foundation under Grant MCS77-27293. The work of G. S. Ho was done while he was at the University of California, Berkeley, CA.

C. V. Ramamoorthy is with the Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, CA 94720.

G. S. Ho is with Bell Laboratories, Naperville, IL 60540.